

<<코드 리뷰>>

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <pthread.h>
#include <string.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <arpa/inet.h>
#include <errno.h>
#include <time.h>

#define PORT 8090 // 서버가 사용할 포트 번호
#define MAX_BUFFER_SIZE 2000 // 요청 버퍼의 최대 크기
#define INITIAL_THREAD_POOL_SIZE 10 // 초기 스레드 풀의 크기
#define NUMBER_OF_POOLS 1 // 초기 설정을 위한 스레드 풀의 수
#define MONITOR_INTERVAL 5 // 모니터링 간격(초 단위)
```

서버는 POSIX 스레드(pthread)를 사용하여 멀티스레딩을 구현하고, 소켓 프로그래밍을 위해 필요한 라이브러리를 포함함.

```
typedef struct {
    int client_socket;
} Request;

typedef struct {
    Request buffer[MAX_BUFFER_SIZE];
    int head;
    int tail;
    int size;
    pthread_mutex_t lock;
    pthread_cond_t not_empty;
} RequestBuffer;

typedef struct {
    pthread_t *threads;
    int size;
    int active;
    RequestBuffer request_buffer;
} ThreadPool;

typedef struct {
    ThreadPool *thread_pools;
    int num_pools;
    int next_pool;
    pthread_mutex_t lock;
} ThreadPoolManager;
```

- 요청(Request), 요청 버퍼(RequestBuffer), 스레드 풀(ThreadPool), 스레드 풀 관리자(ThreadPoolManager) 구조체를 정의합니다.
- 각 구조체는 클라이언트 소켓, 요청 버퍼, 스레드 배열, 스레드 풀 크기, 활성 상태, 스레드 풀 배열 등을 포함합니다.

```
void init_request_buffer(RequestBuffer *buffer) {
    buffer->head = 0;
    buffer->tail = 0;
    buffer->size = 0;
    pthread_mutex_init(&buffer->lock, NULL);
    pthread_cond_init(&buffer->not_empty, NULL);
}
```

요청 버퍼를 초기화하는 함수입니다. 버퍼의 인덱스와 크기를 초기화하고 뮤텍스와 조건 변수를 초기화합니다

```
int is_buffer_full(RequestBuffer *buffer) {
    return buffer->size == MAX_BUFFER_SIZE;
}

int is_buffer_empty(RequestBuffer *buffer) {
    return buffer->size == 0;
}
```

요청 버퍼가 가득 찼는지 또는 비었는지 확인하는 함수입니다

```
void enqueue_request(RequestBuffer *buffer, int client_socket, int
pthread_mutex_lock(&buffer->lock);
while (is_buffer_full(buffer)) {
    pthread_cond_wait(&buffer->not_empty, &buffer->lock);
}
buffer->buffer[buffer->tail].client_socket = client_socket;
buffer->tail = (buffer->tail + 1) % MAX_BUFFER_SIZE;
buffer->size++;
pthread_cond_signal(&buffer->not_empty);
pthread_mutex_unlock(&buffer->lock);

if (client_socket != -1) {
    pthread_mutex_lock(&log_mutex);
    printf("Request enqueued in buffer of pool %d. Current size
pthread_mutex_unlock(&log_mutex);
}
}

int dequeue_request(RequestBuffer *buffer) {
    pthread_mutex_lock(&buffer->lock);
    while (is_buffer_empty(buffer)) {
        pthread_cond_wait(&buffer->not_empty, &buffer->lock);
    }
    int client_socket = buffer->buffer[buffer->head].client_socket;
    buffer->head = (buffer->head + 1) % MAX_BUFFER_SIZE;
    buffer->size--;
    pthread_cond_signal(&buffer->not_empty);
    pthread_mutex_unlock(&buffer->lock);
    return client_socket;
}
```

- 요청을 버퍼에 추가하거나 버퍼에서 제거하는 함수입니다. 추가할 때는 버퍼가 가득 차지 않았는지 확인하고, 제거할 때는 버퍼가 비어 있지 않은지 확인합니다.

```

void init_thread_pool(ThreadPool *pool, int size) {
    pool->size = size;
    pool->active = 1;
    pool->threads = (pthread_t *)malloc(size * sizeof(pthread_t));
    if (pool->threads == NULL) {
        perror("Failed to allocate memory for threads");
        exit(EXIT_FAILURE);
    }
    init_request_buffer(&pool->request_buffer);
}

void *thread_function(void *arg) {
    RequestBuffer *buffer = (RequestBuffer *)arg;
    while (1) {
        int client_socket = dequeue_request(buffer);
        if (client_socket == -1) {
            break;
        }

        struct timespec start_time, end_time;
        clock_gettime(CLOCK_MONOTONIC, &start_time);

        sleep(1.99);

        char buffer[30000] = {0};
        read(client_socket, buffer, 30000);
        pthread_mutex_lock(&log_mutex);
        printf("%s\n", buffer);
        pthread_mutex_unlock(&log_mutex);
        char *hello = "HTTP/1.1 200 OK\nContent-Type: text/plain"
            "Content-Length: 20\n\nMy first web server!";
        write(client_socket, hello, strlen(hello));
        close(client_socket);

        clock_gettime(CLOCK_MONOTONIC, &end_time);

        double task_time = (end_time.tv_sec - start_time.tv_sec) +
            (end_time.tv_nsec - start_time.tv_nsec) / 1e9;

        pthread_mutex_lock(&log_mutex);
        printf("Task time: %.6f seconds\n", task_time);
        pthread_mutex_unlock(&log_mutex);
    }
    return NULL;
}

```

- 스레드 풀을 초기화하는 함수와 각 스레드가 실행할 함수입니다. 스레드는 요청을 처리하고, 클라이언트에 응답을 보낸 후 소켓을 닫습니다.

```

void start_thread_pool(ThreadPool *pool) {
    for (int i = 0; i < pool->size; i++) {
        pthread_create(&pool->threads[i], NULL, thread_function, (void *)&pool->1
    }
}

void gracefully_shutdown_thread_pool(ThreadPool *pool) {
    pool->active = 0;

    for (int i = 0; i < pool->size; i++) {
        enqueue_request(&pool->request_buffer, -1, -1);
    }

    for (int i = 0; i < pool->size; i++) {
        pthread_join(pool->threads[i], NULL);
    }

    free(pool->threads);
    pthread_mutex_destroy(&pool->request_buffer.lock);
    pthread_cond_destroy(&pool->request_buffer.not_empty);
}

void stop_thread_pool(ThreadPool *pool) {
    gracefully_shutdown_thread_pool(pool);
}

```

- 스레드 풀을 시작하고 종료하는 함수입니다. `gracefully_shutdown_thread_pool` 함수는 스레드 풀을 정상적으로 종료합니다.

```

void init_thread_pool_manager(ThreadPoolManager *manager, int num_pools) {
    pthread_mutex_init(&manager->lock, NULL);
    pthread_mutex_lock(&manager->lock);
    manager->num_pools = num_pools;
    manager->next_pool = 0;
    manager->thread_pools = (ThreadPool *)malloc(num_pools * sizeof(ThreadPool));
    if (manager->thread_pools == NULL) {
        perror("Failed to allocate memory for thread pools");
        pthread_mutex_unlock(&manager->lock);
        exit(EXIT_FAILURE);
    }
    for (int i = 0; i < num_pools; i++) {
        init_thread_pool(&manager->thread_pools[i], pool_size);
        start_thread_pool(&manager->thread_pools[i]);
    }
    pthread_mutex_unlock(&manager->lock);
}

void add_thread_pool(ThreadPoolManager *manager, int pool_size) {
    pthread_mutex_lock(&manager->lock);
    ThreadPool* temp = realloc(manager->thread_pools, (manager->num_pools + 1) * sizeof(ThreadPool));
    if (temp == NULL) {
        pthread_mutex_unlock(&manager->lock);
        pthread_mutex_lock(&log_mutex);
        printf("Failed to add new ThreadPool. Memory allocation failed\n");
        pthread_mutex_unlock(&log_mutex);
        return;
    }
    manager->thread_pools = temp;
    init_thread_pool(&manager->thread_pools[manager->num_pools], pool_size);
    start_thread_pool(&manager->thread_pools[manager->num_pools]);
    manager->num_pools++;
}

```

```

manager->num_pools++;
pthread_mutex_unlock(&manager->lock);

pthread_mutex_lock(&log_mutex);
printf("Added new ThreadPool. Total pools: %d\n", manager->num_pools);
pthread_mutex_unlock(&log_mutex);
}

void remove_thread_pool(ThreadPoolManager *manager, int pool_index) {
    pthread_mutex_lock(&manager->lock);
    if (pool_index >= 0 && pool_index < manager->num_pools) {
        gracefully_shutdown_thread_pool(&manager->thread_pools[pool_index]);

        for (int i = pool_index; i < manager->num_pools - 1; i++)
            manager->thread_pools[i] = manager->thread_pools[i + 1];

        manager->num_pools--;
        ThreadPool* temp = realloc(manager->thread_pools, manager->num_pools * sizeof(ThreadPool));
        if (temp == NULL && manager->num_pools > 0) {
            perror("Failed to shrink the memory allocation for thread pools");
            pthread_mutex_unlock(&manager->lock);
            exit(EXIT_FAILURE);
        }
        manager->thread_pools = temp;
    }
    pthread_mutex_unlock(&manager->lock);

    pthread_mutex_lock(&log_mutex);
    printf("Removed a ThreadPool. Total pools: %d\n", manager->num_pools);
    pthread_mutex_unlock(&log_mutex);
}

```

- 스레드 풀 관리자를 초기화하고, 스레드 풀을 추가하거나 제거하는 함수입니다.

```

void enqueue_request_round_robin(ThreadPoolManager *manager, int client_socket) {
    pthread_mutex_lock(&manager->lock);
    int pool_index = manager->next_pool;
    ThreadPool *pool = &manager->thread_pools[pool_index];
    printf("Request being assigned to ThreadPool %d\n", pool_index);
    manager->next_pool = (manager->next_pool + 1) % manager->num_pools;
    pthread_mutex_unlock(&manager->lock);
    enqueue_request(&pool->request_buffer, client_socket, pool_index);
}

void cleanup_thread_pool_manager(ThreadPoolManager *manager) {
    pthread_mutex_lock(&manager->lock);
    for (int i = 0; i < manager->num_pools; i++) {
        stop_thread_pool(&manager->thread_pools[i]);
    }
    free(manager->thread_pools);
    pthread_mutex_unlock(&manager->lock);
}

```

- 요청을 라운드 로빈 방식으로 스레드 풀에 추가하고, 스레드 풀 관리자를 정리하는 함수입니다.

```

void *monitor_and_adjust_pools(void *arg) {
    ThreadPoolManager *manager = (ThreadPoolManager *)arg;
    while (1) {
        int total_pending_requests = 0;
        pthread_mutex_lock(&manager->lock);
        for (int i = 0; i < manager->num_pools; i++) {
            pthread_mutex_lock(&manager->thread_pools[i].request_buffer.lock);
            total_pending_requests += manager->thread_pools[i].request_buffer.pending_requests;
            pthread_mutex_unlock(&manager->thread_pools[i].request_buffer.lock);
        }
        pthread_mutex_unlock(&manager->lock);

        pthread_mutex_lock(&log_mutex);
        printf("Total pending requests: %d\n", total_pending_requests);
        pthread_mutex_unlock(&log_mutex);

        int required_pools = (total_pending_requests + 7) / 8;

        pthread_mutex_lock(&log_mutex);
        printf("Required pools: %d, Current pools: %d\n", required_pools, manager->num_pools);
        pthread_mutex_unlock(&log_mutex);

        while (required_pools > manager->num_pools) {
            add_thread_pool(manager, INITIAL_THREAD_POOL_SIZE);
        }

        while (required_pools < manager->num_pools && manager->num_pools > 1) {
            remove_thread_pool(manager, manager->num_pools - 1);
        }

        sleep(MONITOR_INTERVAL);
    }
}

```

- 스레드 풀의 수를 모니터링하고 필요한 경우 조정하는 함수입니다. 총 대기 중인 요청 수에 따라 스레드 풀을 추가하거나 제거합니다.

<<특이사항>>

진척도에서 조금 아쉬움이 있는 코드입니다. 실제로 구현을 했지만 request의 bursty한 모드에서 MLQ처럼 buffer안의 request의 개수에 맞추어 threadPool을 병렬적으로 증가시키는 것에는 buffer의 총 request 수가 "8의 배수 단위로 증가 할 때 threadPool의 개수가 늘어나도록 설정" 함으로써(80%에 대응되는 로직) 성공했습니다만, 이후 threadPool을 제거하는 로직에서 실행이 안되는 조금 아쉬움이 많이 나는 작업이었습니다. (여러 디버깅을 거듭 한 결과 임에도 불구하고.... 아마 이거는 제 공부의 부족이라서 크게 할 말이 없네요) 20%이하를 감지하는 로직을 부드럽게 구현하지를 못했습니다.